The Ozaki Scheme

Emulating FP64 matrix multiplication on low-precision hardware

A. J. Byrnes
University of Illinois Chicago

2026-11-03

Intro

Why the Ozaki scheme?

- HPC ranks itself by double-precision LINPACK (TOP500)
- Accelerator hardware is shifting toward low-precision AI throughput
- **Emulation:** get FP64-accurate results out of low-precision units

Foundations

Floating-point basics

$$\text{value} = \text{mantissa} \times 2^{\text{exponent}}$$

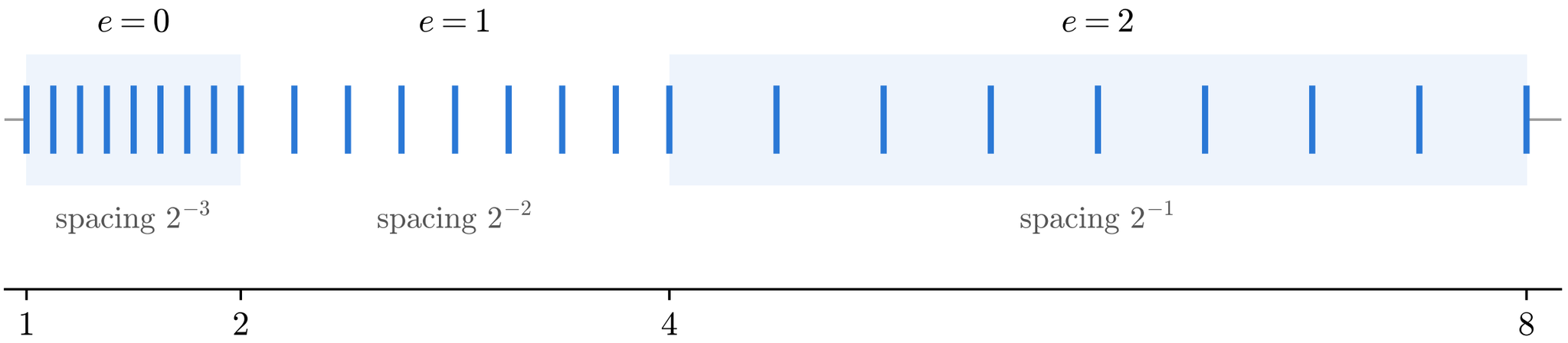
$$\text{e.g.} \quad 28.016 = 1.751 \times 2^4$$

Our discussion is concerned with FP64:

- 53-bit mantissa
- 11-bit exponent

Max relative error from rounding a real number to FP64:

Floating-point space

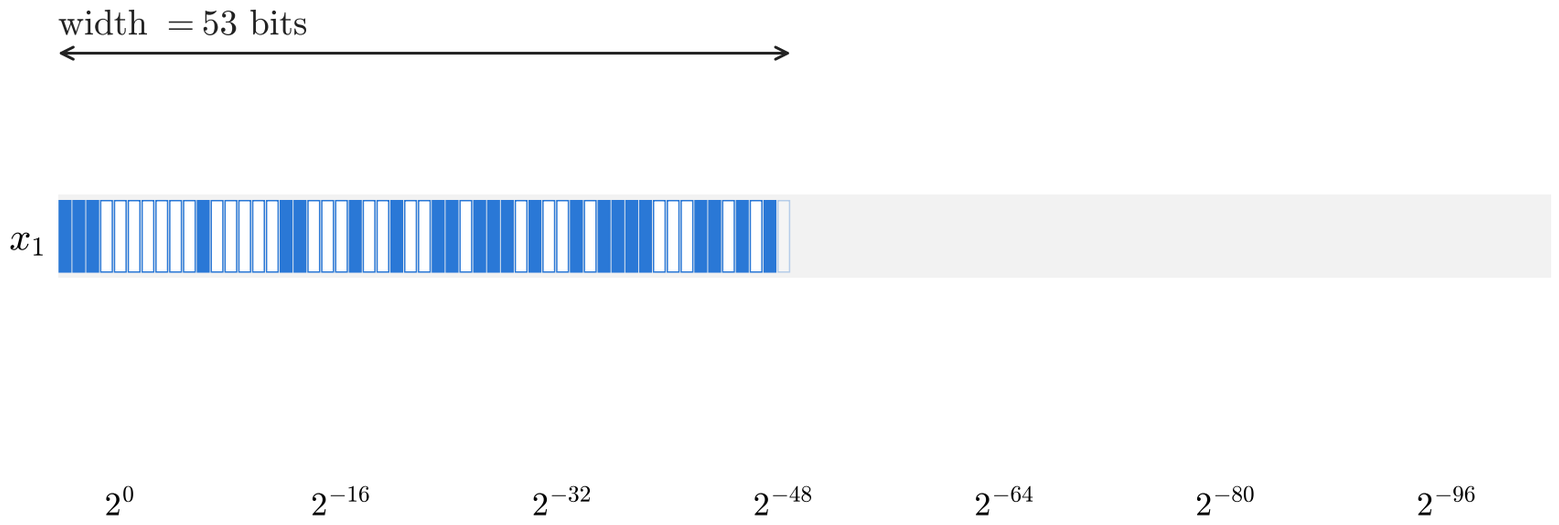


- **Floating-point space:** Floating-point values are a subset of points on the real number line.
- Density of coverage decreases as magnitude grows

Density decreases as the exponent grows.

Mantissa space: One value

$$x_1 = 1.751 \times 2^4$$



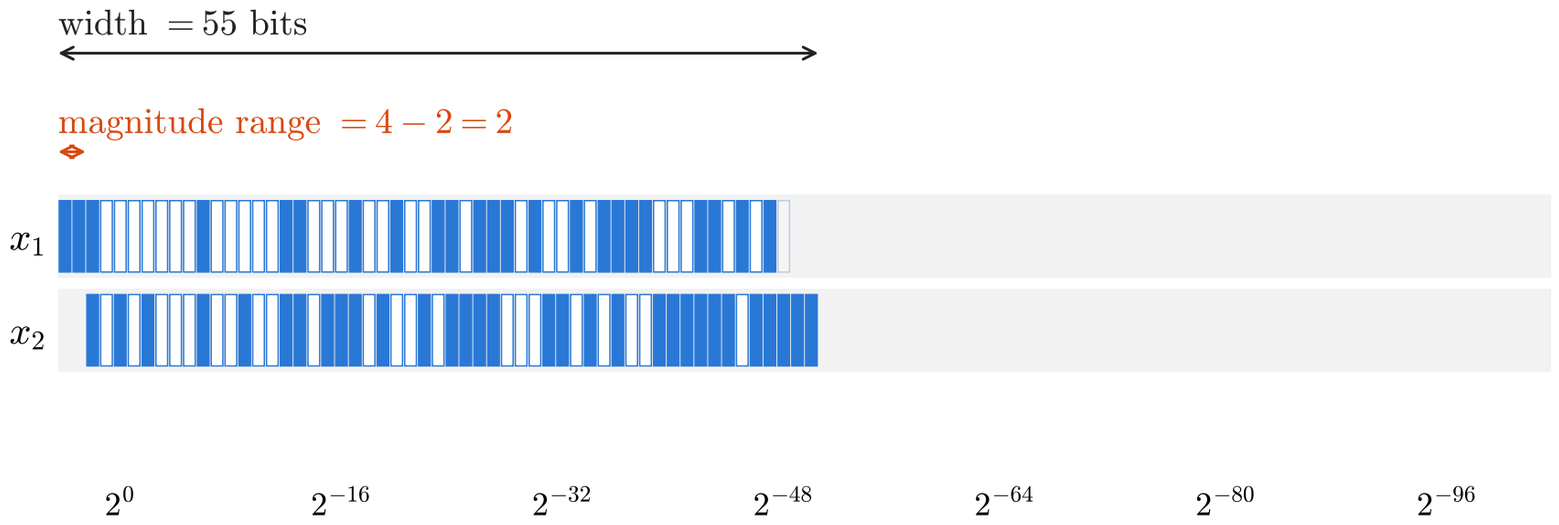
Mantissa space: Visualization of floating-point mantissas, showing the magnitude of each bit

One floating-point value: Mantissa space is 53 bits wide.

Mantissa space: Two values

$$x_1 = 1.751 \times 2^4$$

$$x_2 = 1.317 \times 2^2$$



Width: Distance from the most significant bit of the highest-magnitude value to the least significant bit of the lowest-magnitude value.

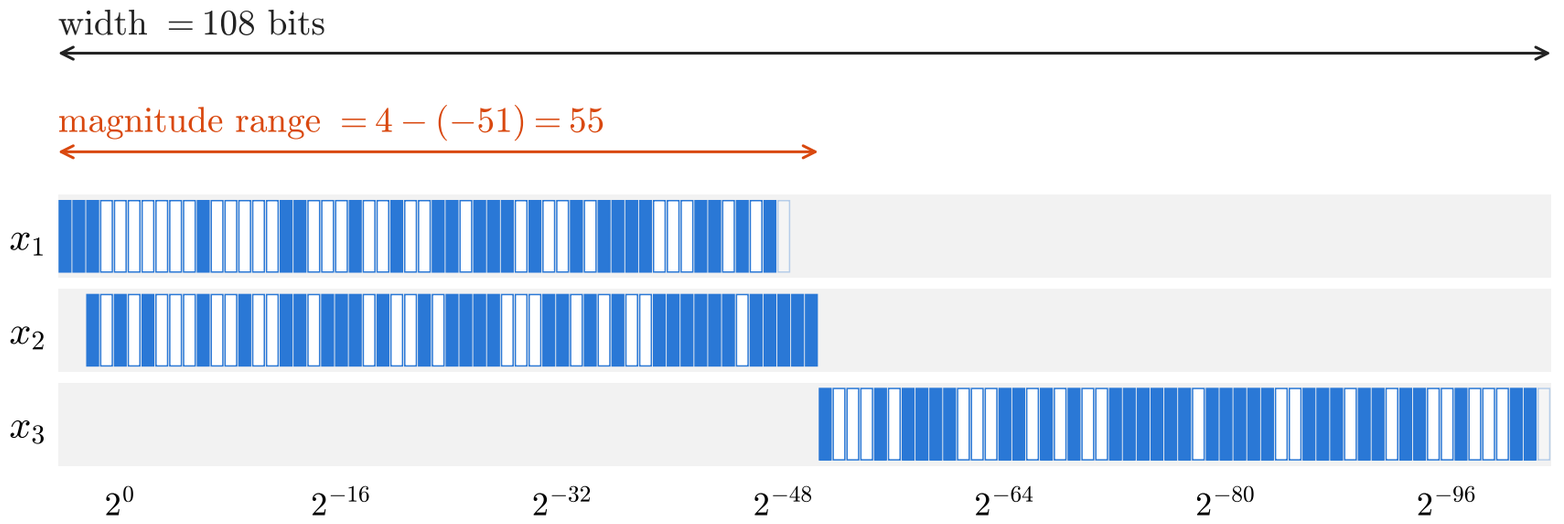
Magnitude range: Difference in exponents of the highest- and lowest-magnitude values.

Mantissa space: Three values

$$x_1 = 1.751 \times 2^4$$

$$x_2 = 1.317 \times 2^2$$

$$x_3 = 1.092 \times 2^{-51}$$



x_1 and x_2: Magnitude range = 2, mantissa space width = 55

x_1, x_2, and x_3: Magnitude range = 55, mantissa space width = 108

Larger magnitude range → Wider mantissa space

The Ozaki Scheme

Rounding

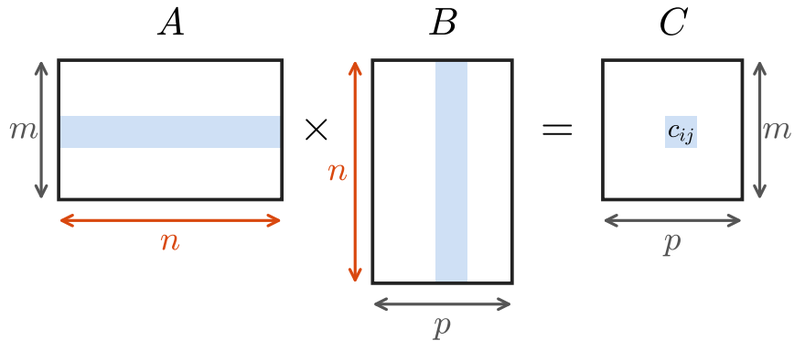
IEEE 754: Primitive operations should have results as if they were performed exactly, and then rounded to FP64.

$$\text{max relative error of a single FLOP} = 1.11 \times 10^{-16}$$

Error accumulates

Rounding error can accumulate over *sequences* of floating-point operations.

For a matrix product, the max error grows with the inner dimension n.



Each entry c_{ij} is a dot product of two length-n vectors.

Reducing rounding error

There have been many different methods for attempting to reduce error.

Multiple-precision libraries

e.g. MPFR, XBLAS

(Bailey 1995; Li et al. 2002; *The MPFR Library* 2010; Fujiwara 2010)

Accurate summation and dot product algorithms

e.g. compensated sums

(Demmel and Hida 2003; Ogita et al. 2005; Rump et al. 2008a, 2008b)

These methods are slower than straightforward FP64 arithmetic.

Ozaki et al. (2012)

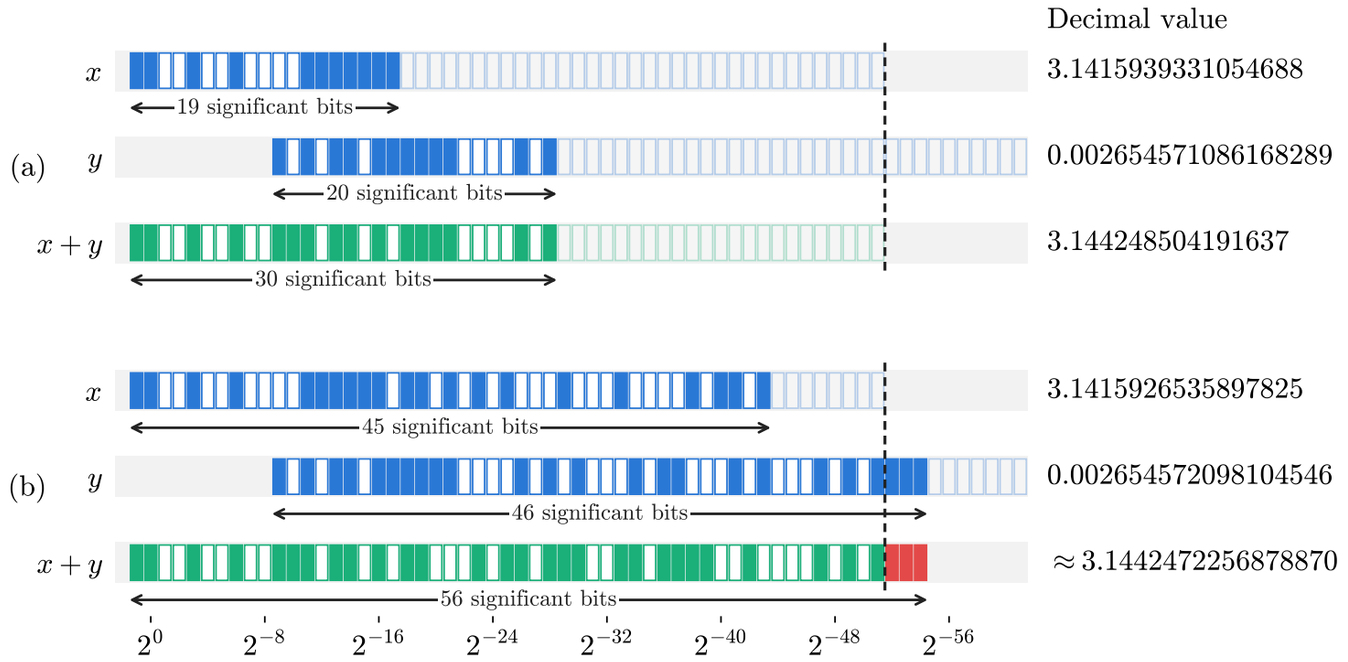
Error-free transformations of matrix multiplication by using fast routines of matrix multiplication and its applications

Katsuhisa Ozaki Shibaura Institute of Technology
Takeshi Ogita Tokyo Woman's Christian University
Shin'ichi Oishi Waseda University
Siegfried M. Rump Waseda University; Hamburg University of Technology

Published in *Numerical Algorithms*, 2012 (Ozaki et al. 2012)

Fast routine for computing matrix products with reduced rounding error

Rounding in mantissa space



The result is rounded only if it doesn’t fit in the 53-bit mantissa.

If the operands have few enough significant bits, the result is exact.

Splitting

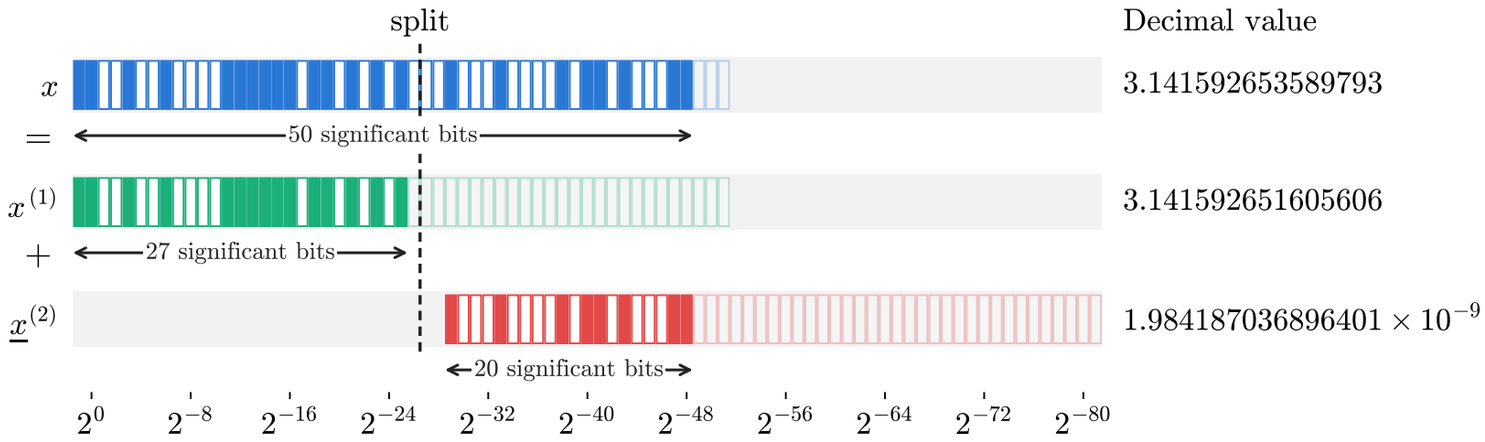
Ozaki et al. build on a method from Rump et al. (2008) for splitting a floating-point value into high- and low-magnitude parts (Rump et al. 2008a).

$$x = x^{(1)} + \underline{x}^{(2)}$$

$$x^{(1)} = \mathrm{fl}\Big((\sigma + x) - \sigma\Big) \qquad \underline{x}^{(2)} = \mathrm{fl}(x - x^{(1)})$$

Splitting factor: $\sigma = 2^M \times 2^{\lceil \log_2 |x| \rceil}$, a power of 2 that is M bits above x

Splitting in mantissa space



Adding σ pushes the lowest M bits of x off the end of the mantissa, so they're rounded away.

Every slice $x^{(1)}$ has at most $53 - M$ significant bits, and the remainder $\underline{x}^{(2)}$ keeps the rest.

Exact products

If we split x and y so that the slices have few enough significant bits, then $x^{(1)}y^{(1)}$ will be exact.

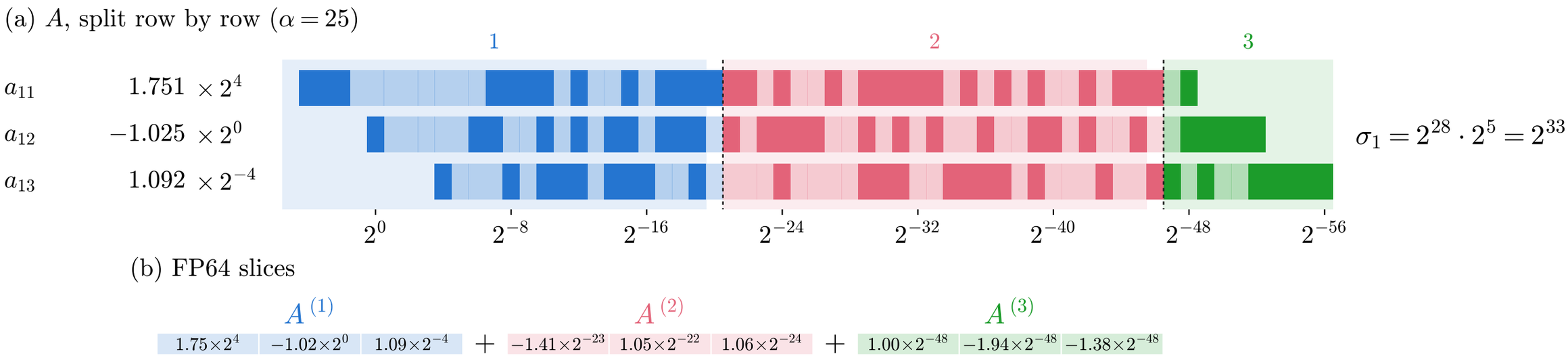
If we kept applying this procedure to the remainders, we could compute the product xy as a sum of exact products $x^{(i)}y^{(j)}$:

$$xy = \sum_{i=1}^{n_x} \sum_{j=1}^{n_y} x^{(i)} y^{(j)}$$

The Ozaki scheme

1. **Decompose:** Iteratively split A and B into slices
2. **Multiply:** Compute slice and remainder products
3. **Accumulate:** Sum slice and remainder products

Decompose



Each row of A gets its own splitting factor, scaled to the row's largest element.

Decompose: Full or partial

Full decomposition: Split until the remainder is zero

$$A = A^{(1)} + \cdots + A^{(n_A)} \qquad B = B^{(1)} + \cdots + B^{(n_B)}$$

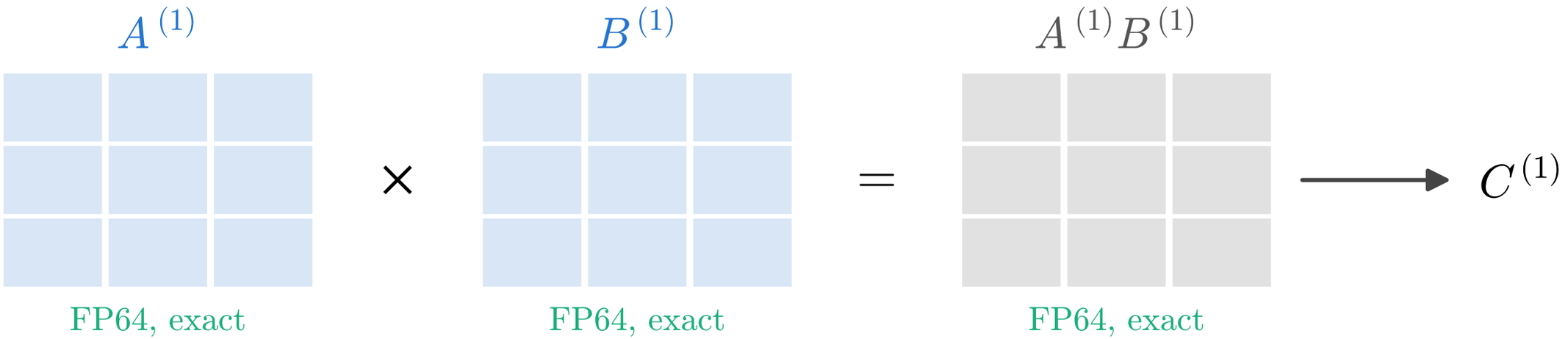
$$AB = \sum_{i=1}^{n_A} \sum_{j=1}^{n_B} \underbrace{A^{(i)} B^{(j)}}_{\text{exact}}$$

Partial decomposition: Stop after $k - 1$ slices

$$A = A^{(1)} + \cdots + A^{(k-1)} + \underline{A}^{(k)} \qquad B = B^{(1)} + \cdots + B^{(k-1)} + \underline{B}^{(k)}$$

$$AB = \underbrace{\sum_{i+j\leq k} A^{(i)} B^{(j)}}_{\text{exact}} + \underbrace{\sum_{i=1}^{k-1} A^{(i)} \underline{B}^{(k-i+1)} + \underline{A}^{(k)} B}_{\text{low-magnitude remainders}}$$

Multiply



Each slice product is a standard FP64 matrix multiplication, and it's exact.

(Products involving remainders are also standard FP64 matrix multiplication, but not exact.)

Accumulate

Standard FP64 summation

Rounding error only in the additions.

Accurate summation

Little to no rounding error at all.

Result: Matrix product that is more accurate than standard FP64

Impacts of implementation

Optimized BLAS: Every slice and remainder product is a standard FP64 matrix multiplication

BLAS = Basic Linear Algebra Subprograms.

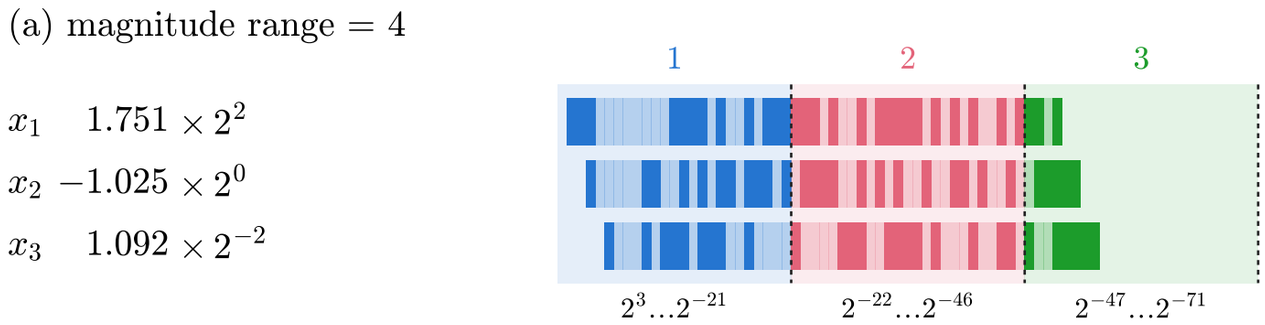
Vendors ship BLAS implementations optimized for their hardware.

Memory overhead: Every slice is a full FP64 matrix

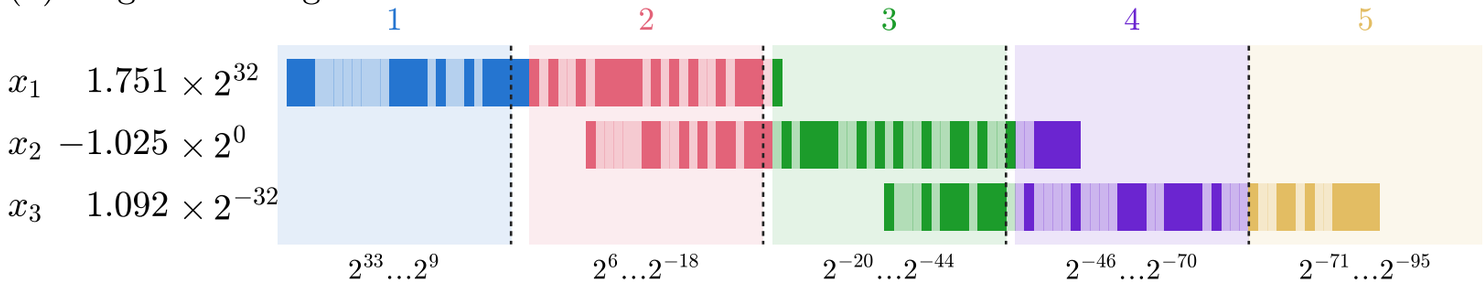
s slices of $A \rightarrow s\times$ the memory of A

Trade-off: Accuracy vs. performance

(a) magnitude range $= 4$



(b) magnitude range $= 64$



Slice width $\alpha = 25$ bits (length-3 vectors)

Slices have to cover the whole mantissa space: Larger magnitude range → More slices → More slice products

This is inherent to the scheme, not to any one implementation

Evaluation

Goal: Show it's a more efficient alternative to *other accurate matrix product methods*

- **Hardware:** Intel Core 2 Duo, Intel Core 2 Extreme
- **Software:** MATLAB
- **Compared against:** FP64 matrix multiplication, `lssresidual` (INTLAB), XBLAS

Test inputs: Matrices of uniform random values, with exponents spread out by ϕ

Evaluation: Results

M1 FP64 · **M2** `lssresidual` · **M3–M5** Ozaki scheme, $k = 2, 3, 4$ slices · **M6** XBLAS

Accuracy (maximum componentwise relative error)

ϕ	M_1	M_2	M_3	M_4	M_5	M_6
1	5.64e–010	8.12e–011	7.95e–015	2.20e–016	3.27e–016	1.11e–016
5	3.48e–011	2.39e–011	7.28e–012	2.19e–016	3.24e–016	1.11e–016
10	2.90e–011	2.68e–011	8.88e–011	1.59e–012	2.21e–014	1.11e–016
15	6.81e–012	5.29e–012	5.39e–012	5.60e–012	4.18e–012	1.11e–016

Performance relative to standard FP64 routines

ϕ	M_1	M_2	M_3	M_4	M_5	M_6
1	1	3.30	3.34	7.41	10.4	99.1
5	1	3.29	3.35	7.62	11.7	95.9
10	1	3.30	2.40	3.41	8.60	84.9
15	1	3.30	2.74	3.81	7.13	97.1

Ozaki scheme: Much faster than XBLAS, with comparable accuracy

More slices → More accurate, but slower

Ozaki on IMMU

Tensor Cores

Tensor Cores: Dedicated matrix multiplication units on NVIDIA GPUs (since 2018)

Low-precision inputs, higher-precision accumulation

First Ozaki scheme on Tensor Cores used FP16-FP32 (Mukunoki et al. 2020)

Ootomo et al. (2024)

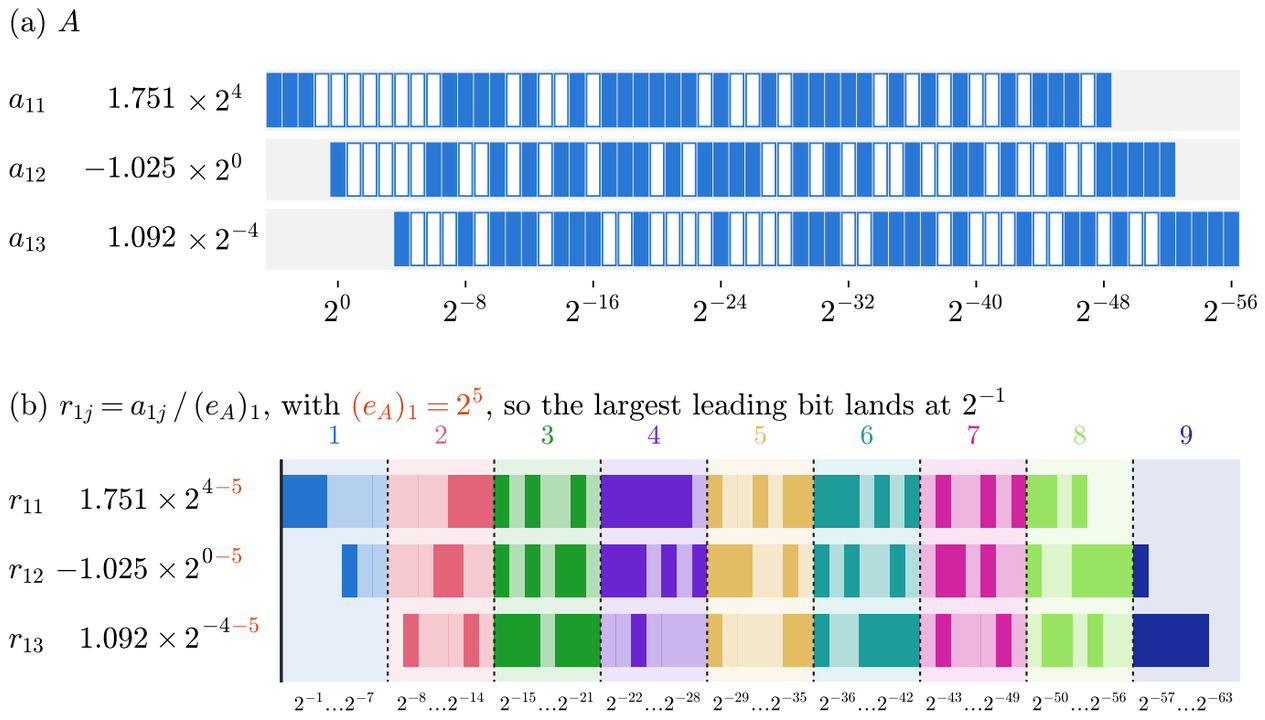
DGEMM on integer matrix multiplication unit

Hiroyuki Ootomo Tokyo Institute of Technology
Katsuhisa Ozaki Shibaura Institute of Technology
Rio Yokota Tokyo Institute of Technology

Published in *The International Journal of High Performance Computing Applications*, 2024 (Ootomo et al. 2024)

Ozaki scheme on integer Tensor Cores (INT8)

Integer slices: Separating magnitude



Problem: INT8 Tensor Cores need integer inputs.

FP64 slices carry per-element magnitude that integers can't hold.

Fix: Separate magnitude from mantissa, so the mantissa fits in INT8.

Rescale each row by a power of 2, $(e_A)_i$, so its largest leading bit lands at 2^{-1}.

Integer slices: Block-float decomposition

(c) INT8 slices, slice p weighted 2^{-7p}

$$
\begin{array}{ccccc}
 & A^{(1)} & & A^{(2)} & \\
2^{-7} \; 2^5 & 112 \;\; -4 \;\; 0 & + \; 2^{-14} \; 2^5 & 7 \;\; -12 \;\; 34 & + \; 2^{-21} \; 2^5 \; 82 \;\; -86 \;\; 119 \\
 & A^{(4)} & & A^{(5)} & \\
+ \; 2^{-28} \; 2^5 & 126 \;\; -117 \;\; 16 & + \; 2^{-35} \; 2^5 & 75 \;\; -114 \;\; 67 & + \; 2^{-42} \; 2^5 \; 117 \;\; -82 \;\; 79 \\
 & A^{(7)} & & A^{(8)} & \\
+ \; 2^{-49} \; 2^5 & 37 \;\; -52 \;\; 34 & + \; 2^{-56} \; 2^5 & 104 \;\; -79 \;\; 52 & + \; 2^{-63} \; 2^5 \; 0 \;\; -64 \;\; 124
\end{array}
$$

$$
A = \underbrace{\operatorname{diag}(e_A)}_{\text{per-row scale}} \sum_{p=1}^{s} \underbrace{2^{-7p}}_{\text{per-slice weight}} \underbrace{A^{(p)}}_{\text{INT8}}
$$

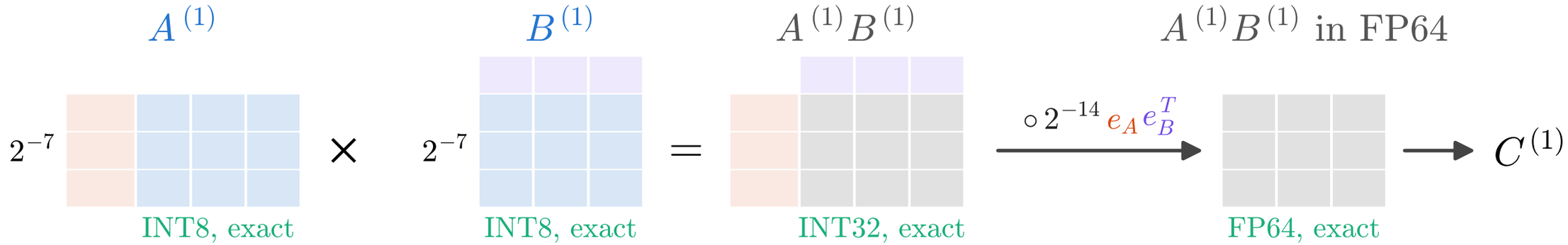
e_A: Per-row scaling vector: Pulls each row's magnitude into one FP64 scalar.

2^{-7p}: Per-slice weight: The magnitude the slice index carries, implicit.

$A^{(p)}$: The actual INT8 matrix: Pure mantissa bits, no magnitude of its own.

e_A and e_B are set aside, and reapplied when the slice products are converted back to FP64.

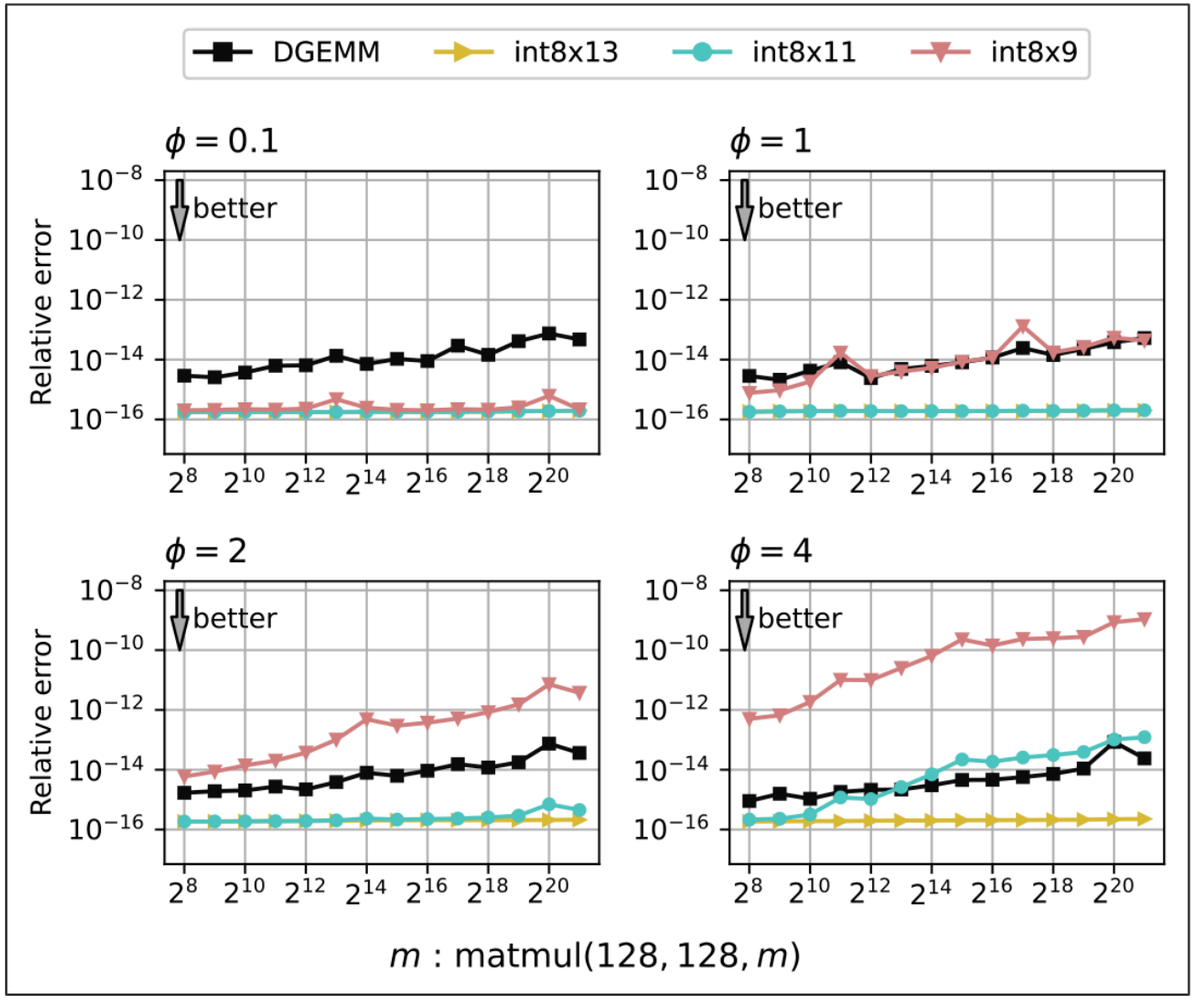
Slice products on integer Tensor Cores



INT8 × INT8 → INT32, exact.

Then convert to FP64, rescale, and accumulate in FP64.

Evaluation: Accuracy



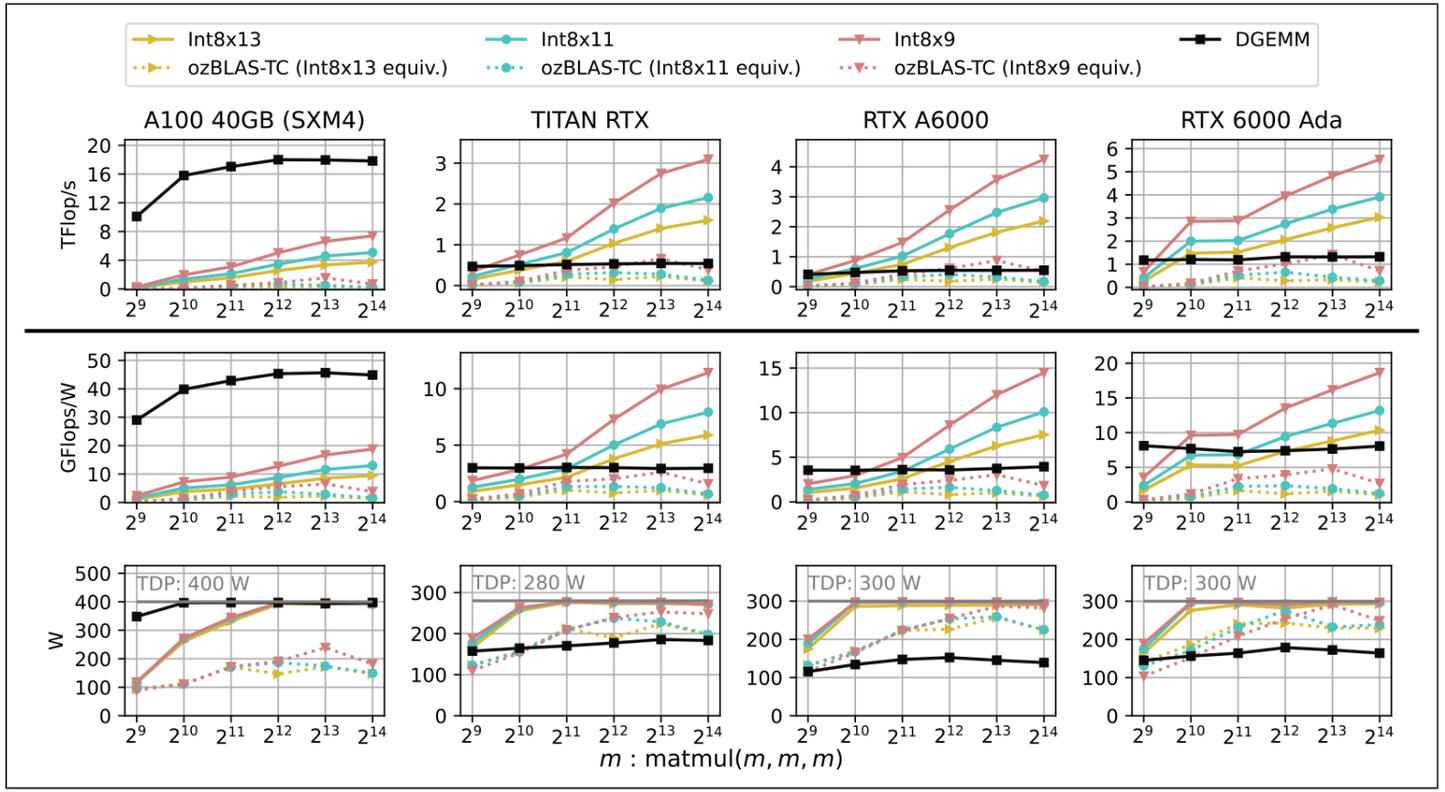
Axes: Max componentwise relative error vs. problem size m

Same synthetic matrices as Ozaki et al. (2012), with $\phi = 0.1, 1, 2, 4$.

INT8x9, x11, x13 = 9, 11, 13 slices.

Larger magnitude range → More slices needed

Evaluation: Performance



Peak INT8 : FP64 throughput: **A100** 32 : 1 · **TITAN RTX** 512 : 1 · **RTX A6000** 512 : 1 · **RTX 6000 Ada** 512 : 1

Faster than native DGEMM on consumer GPUs, but **not** on the A100

Impacts of implementation

Slice width: An INT8 slice holds at most 8 bits (7 + sign).

FP64 slices in our running example held $\alpha = 25$ bits.

$\rightarrow$ More slices ($3 \rightarrow 9$ for our 1×3 A), and more slice products

Memory overhead: But an INT8 slice is 1/8 the size of an FP64 slice.

8 INT8 slices ≈ the storage of 1 FP64 slice

Performance: INT8 throughput may be high enough to outperform FP64.

Usability: Users still have to choose the slice count themselves.

Ozaki with ADP

Emulation in cuBLAS

2025: FP64 emulation added to cuBLAS (NVIDIA 2025)

NVIDIA is leaning on emulation to win back FP64 performance.

Schwarz et al. (2026)

Guaranteed DGEMM Accuracy While Using Reduced Precision Tensor Cores Through Extensions of the Ozaki Scheme

Angelika Schwarz, Anton Anders, Cole Brower, Harun Bayraktar, John Gunnels, Kate Clark, RuQing G. Xu, Samuel Rodriguez, Sebastien Cayrols, Pawel Tabaszewski, Victor Podlozhnyuk

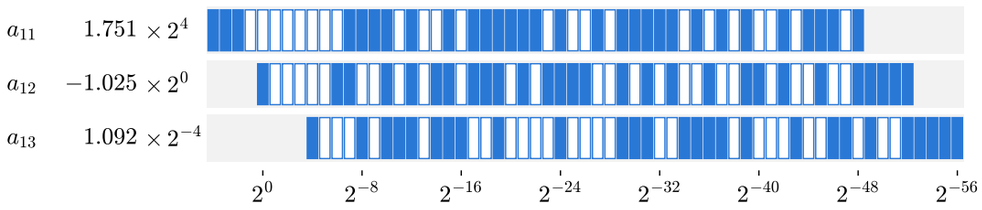
NVIDIA

Published in *SCA/HPCAsia '26*, 2026 (Schwarz et al. 2026)

Use emulation automatically, only when it's accurate and fast

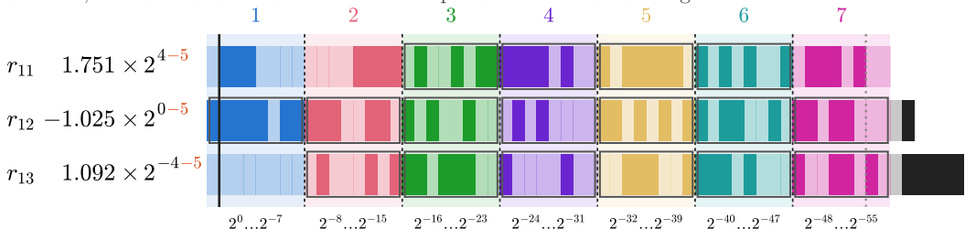
Unsigned slices

(a) A



(b) $r_{1j} = a_{1j} / (e_A)_1$, with $(e_A)_1 = 2^5$, so the largest leading bit lands at 2^{-1}

ESC = 0; boxed: slices re-encoded in two's complement when stored as signed int8

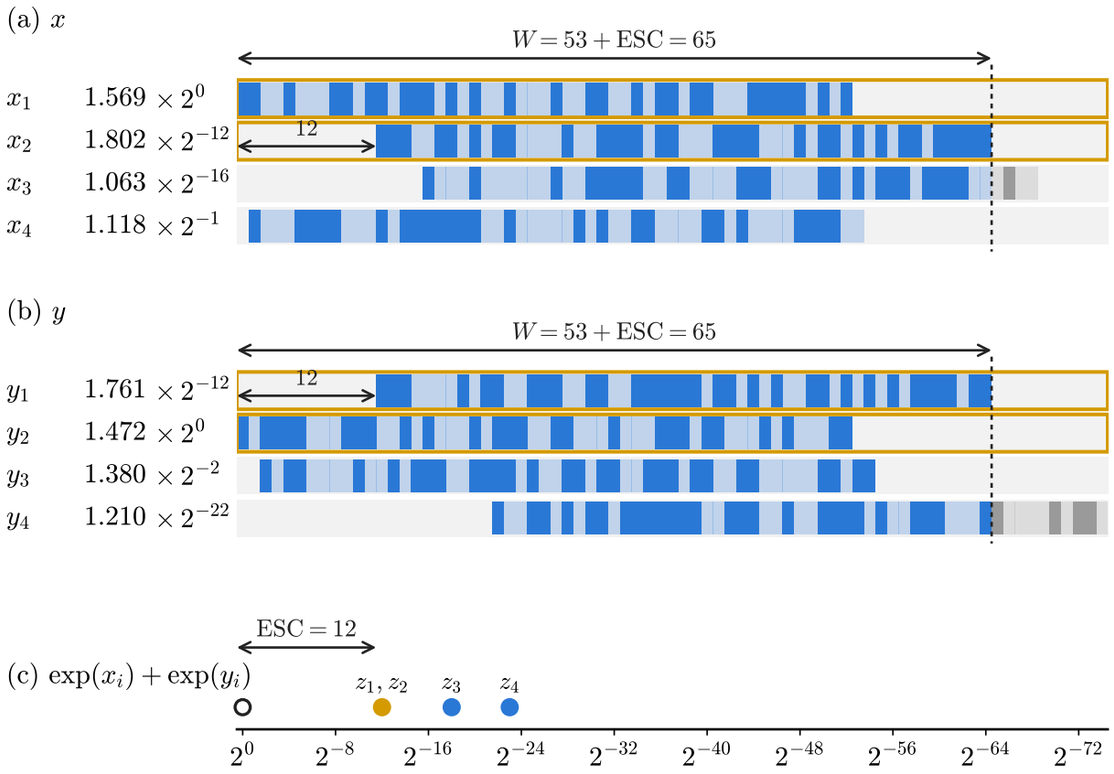


(c) INT8 slices, slice k weighted $2^{-(8k-1)}$

$$2^{-7}\,[2^5\ 112\ -4\ 0] + 2^{-15}\,A^{(2)}[2^5\ 15\ -25\ 70] + 2^{-23}\,A^{(3)}[2^5\ 76\ -92\ -35]$$
$$+ 2^{-31}\,A^{(4)}[2^5\ -11\ 81\ -124] + 2^{-39}\,A^{(5)}[2^5\ -65\ -42\ 58] + 2^{-47}\,A^{(6)}[2^5\ -87\ -77\ -23]$$
$$+ 2^{-55}\,A^{(7)}[2^5\ 116\ -40\ -102]$$

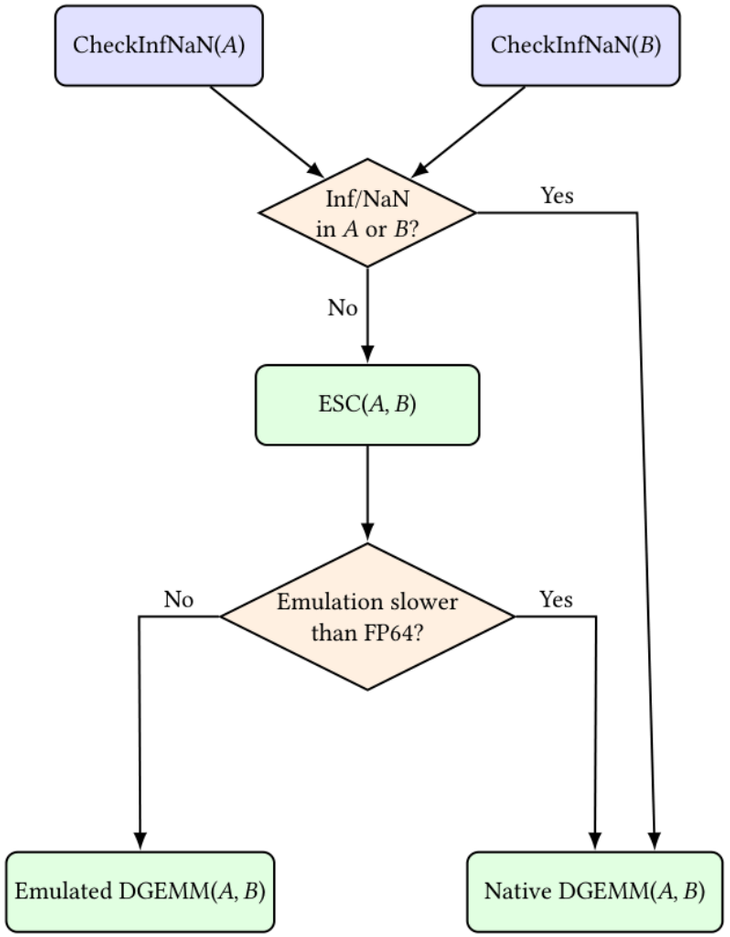
Only the first slice carries the sign, so every other bit holds mantissa.

Exponent span capacity (ESC)



ESC: How much wider than 53 bits the mantissa space has to be, so slices cover it

Automatic Dynamic Precision (ADP)

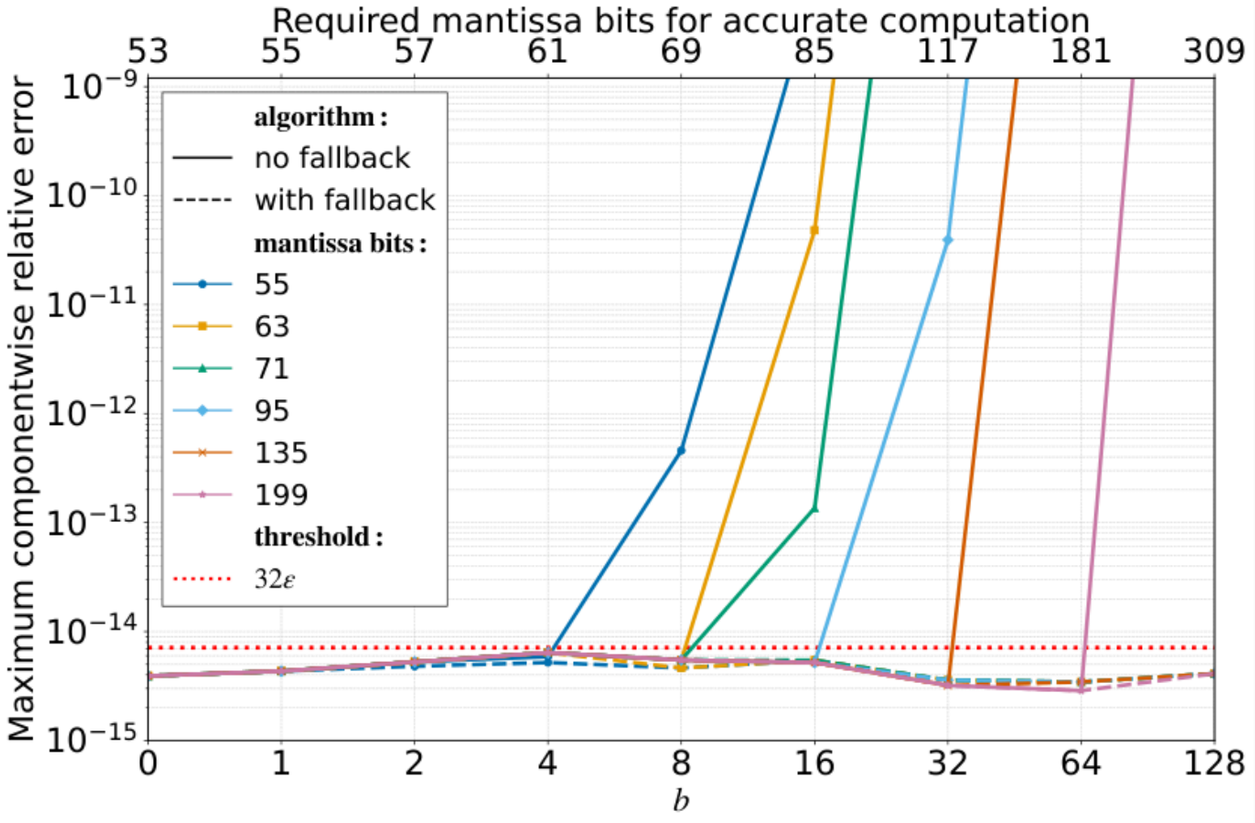


Check for Inf/NaN, compute ESC, then decide: emulate, or fall back to native DGEMM

Goal: Always at least as accurate and as fast as native FP64

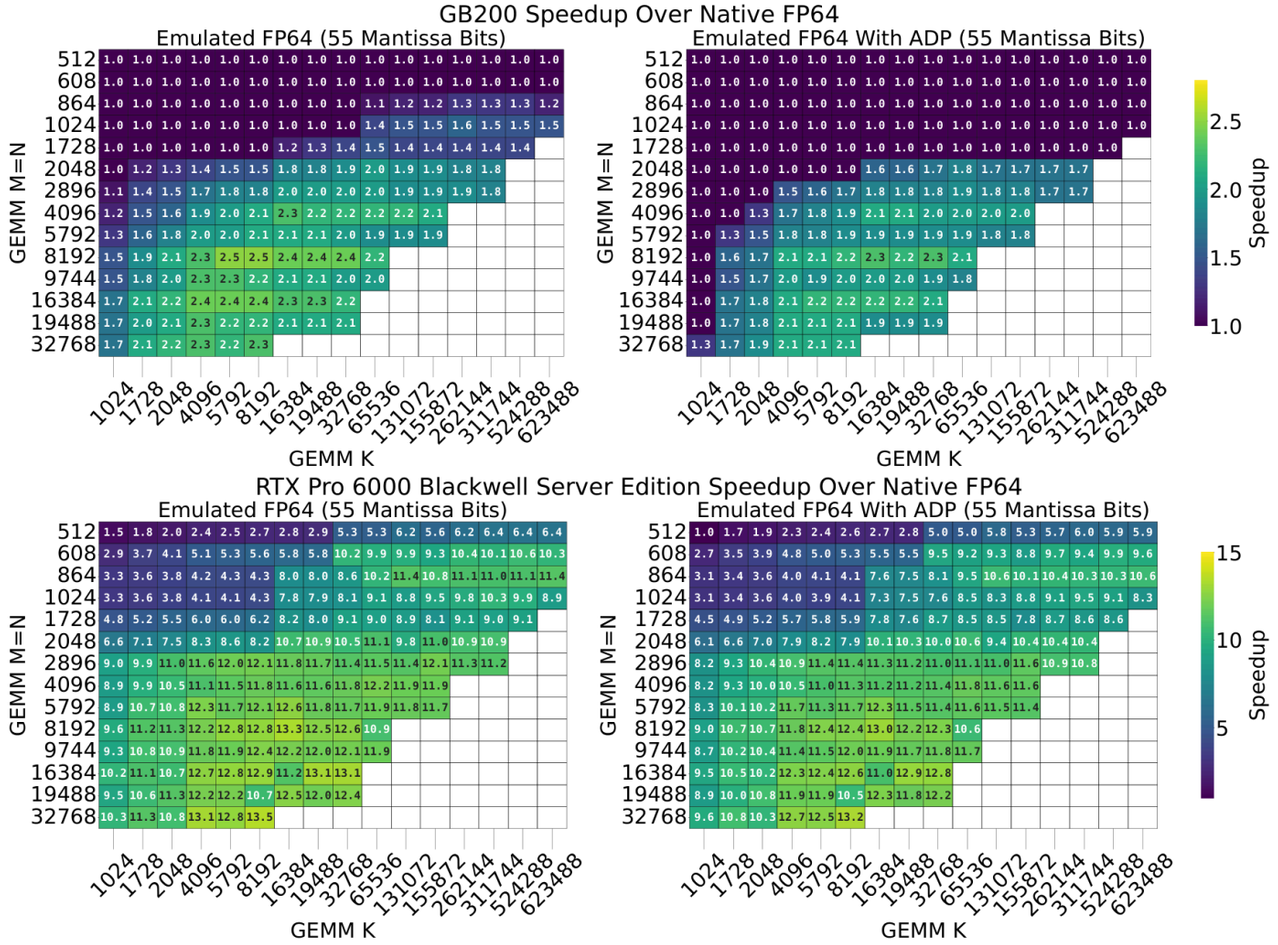
The heuristic for “emulation slower than FP64?” isn’t described.

Evaluation: Accuracy



With fallback: Error stays at native DGEMM level, at every mantissa bit count

Evaluation: Performance



Impacts of implementation

Performance: Same breakdown as Ootomo et al., plus ESC overhead

Representation: Unsigned slices save one bit; little room left to improve

Usability: Users never have to think about magnitude range or hardware…

… but the heuristic that makes this work is a black box

Ozaki in Applications

So far: Synthetic workloads

Every evaluation so far: One matrix product, synthetic inputs

Are there matrix products in real applications that would benefit from emulation?

What happens when emulation runs inside a larger application?

Early examples

Quantum circuit simulation (Ootomo et al. 2024)

2–4.85× faster than cuBLAS ZGEMM, same or better accuracy

(Random unitary circuits: “toy models”)

QR factorization (Schwarz et al. 2026)

Without ADP: 5× faster, error 2.9e-12

With ADP: 3.5× faster, error 7e-16

Brower et al. (2026)

Mixed-Precision Ab Initio Tensor Network State Methods Adapted for NVIDIA Blackwell Technology via Emulated FP64 Arithmetic

Cole Brower, Samuel Rodriguez Bernabeu, Jeff Hammond, John Gunnels NVIDIA
Sotiris S. Xantheas Pacific Northwest National Laboratory; University of Washington
Martin Ganahl SandboxAQ
Andor Menczer, Örs Legeza Wigner Research Centre for Physics

Published in *Journal of Chemical Theory and Computation*, 2026 (Brower et al. 2026)

FP64 emulation in a real quantum chemistry application

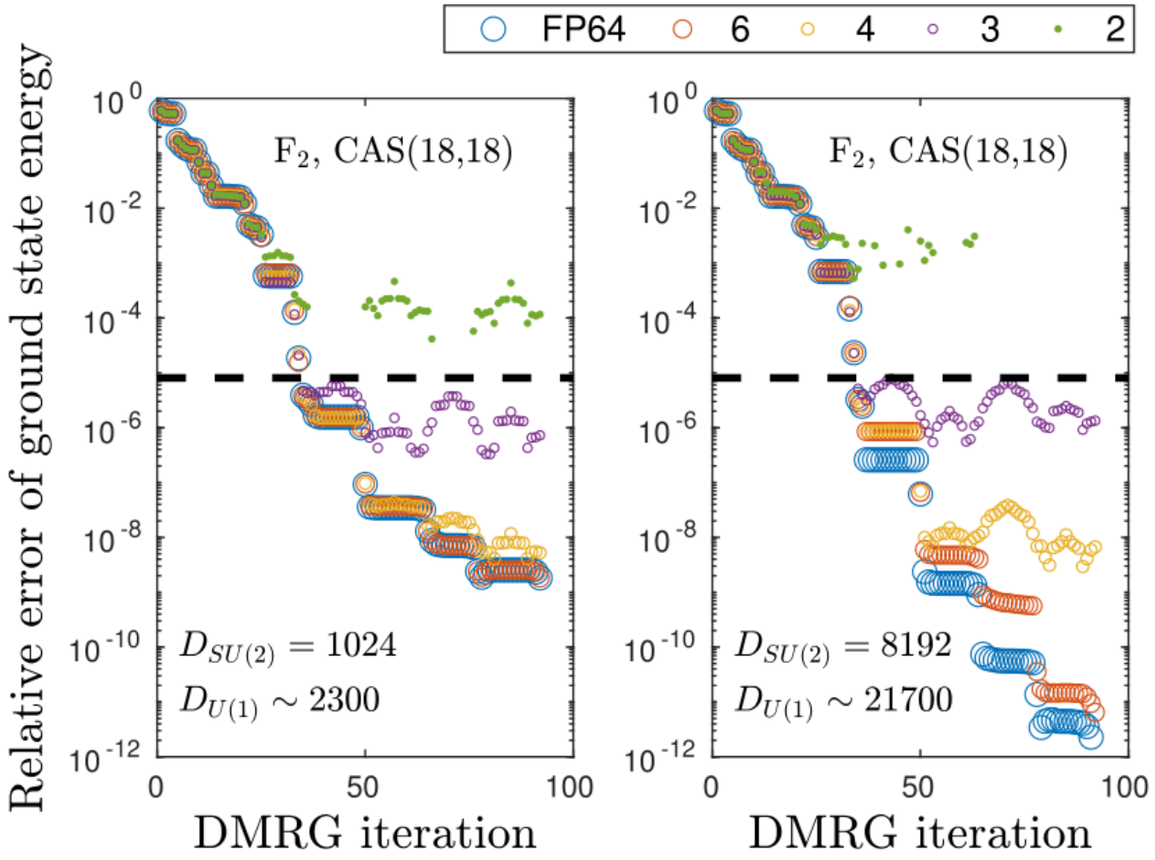
DMRG

DMRG: Iterative algorithm for finding the lowest-energy state of a molecule

Every iteration: Many DGEMMs, some in rounding-sensitive steps (eigensolver, SVD)

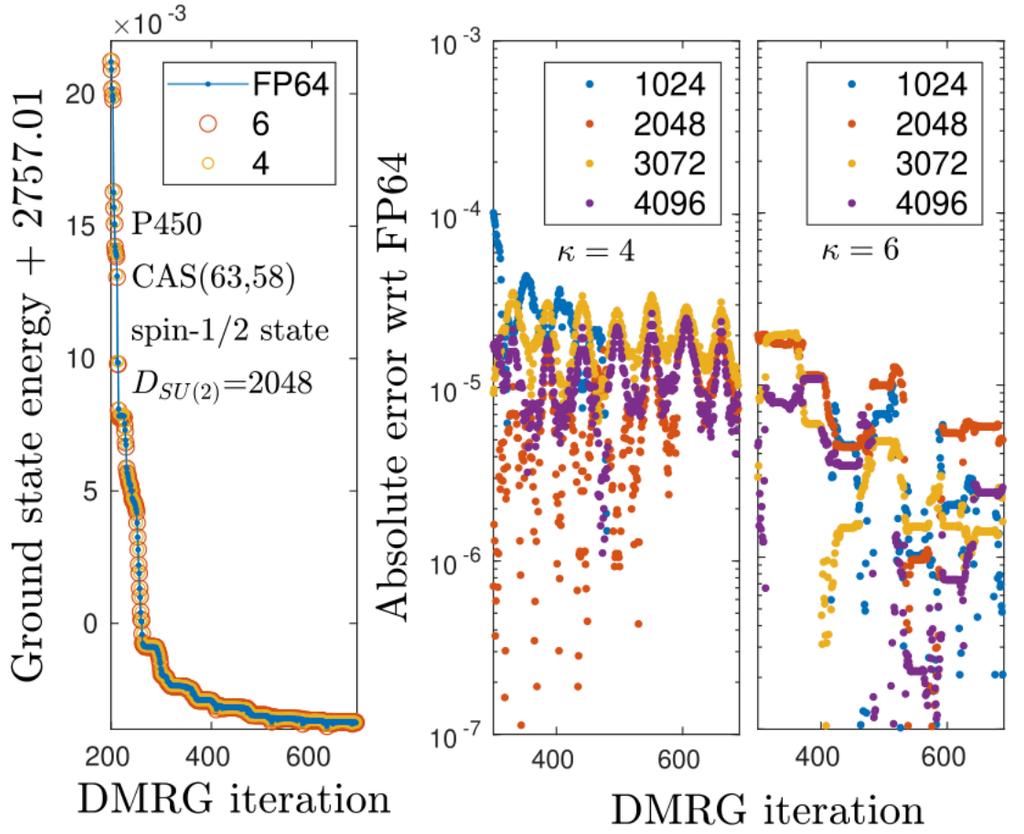
Higher bond dimension $D \rightarrow$ More accurate, but more memory and compute

Accuracy: Small systems



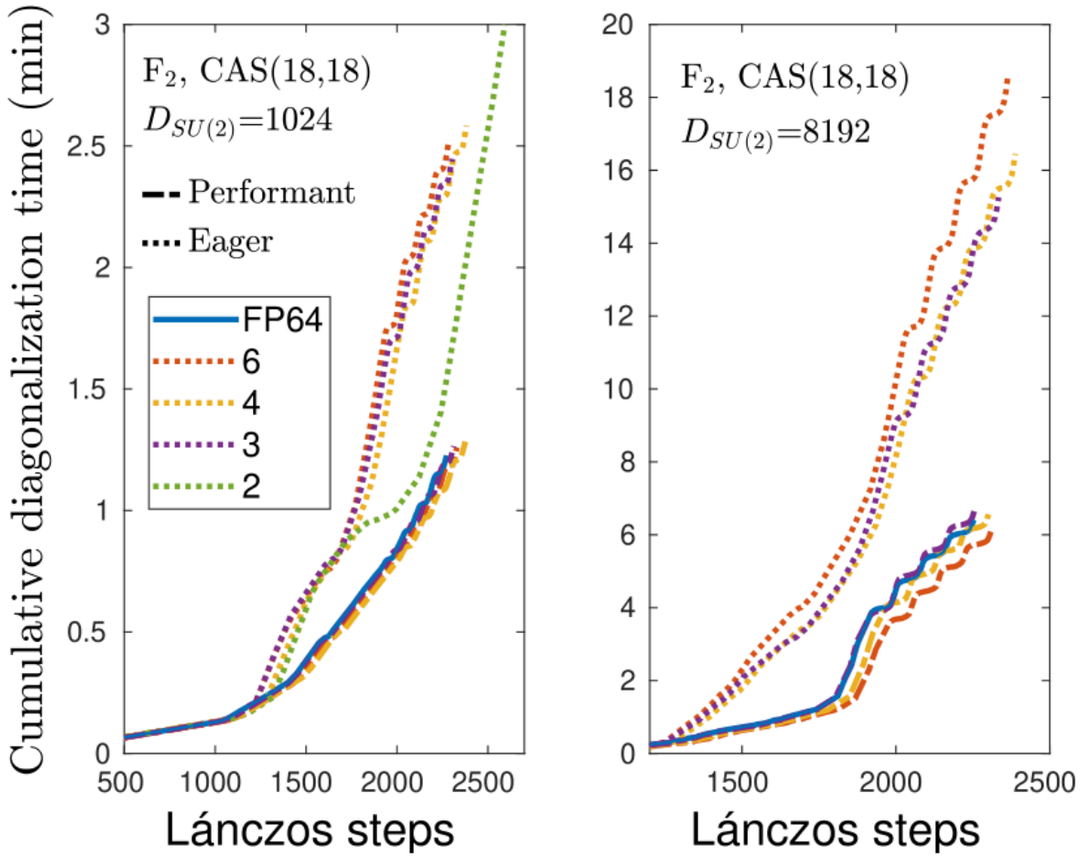
6 and 4 slices track FP64.

Accuracy: Large system



4 and 6 slices agree with FP64 over 700 iterations.

Performance



Eager mode: Emulation is slower than native FP64, even with 2 slices

Discussion

Tracing the Ozaki scheme

Error-free matrix multiplication
Ozaki et al. (2012)

→ Hardware

Integer Tensor Cores
Ootomo et al. (2024)

→ Usability

ADP
Schwarz et al. (2026)

→ Stability

Applications
Brower et al. (2026)

Ozaki (2012) $\rightarrow$ Integer Tensor Cores: Hardware

What changed: Block-float INT8 slices, exact INT32 slice products, FP64 accumulation

None of it makes the scheme do less work: More slices ($3 \rightarrow 9$), more slice products

What changed underneath: 512 : 1 INT8 : FP64 throughput on consumer GPUs, only 32 : 1 on the A100

Nothing about the implementation explains the speedups.

It's a hardware story.

Integer Tensor Cores → ADP: Usability

A strong effort toward usability: ESC picks the slice count, ADP picks emulation or native FP64

But a lot of handwaving:

What is the heuristic that decides whether emulation is performant?

How do we know ESC's slice counts keep FP64-level accuracy?

ADP → Applications: Stability

Stable in a real application: 4–6 slices track FP64 over hundreds of DMRG iterations

But not useful for every application or problem size: Slower than native FP64 on the DMRG problem they timed

Some of that is choosing the wrong problem.

On small problems, emulation just isn't going to help.

What the evidence tells us

Every speedup we've seen is on hardware where FP64 has been deprioritized for AI.

Every accuracy test before DMRG uses synthetic matrices, with one magnitude-range parameter.

What we can conclude: On AI-dedicated hardware, we can get speedups.

Not: Emulation can replace native FP64.

Discussion (continued)

- Practicality: hardware ratio, slice count, ESC overhead
- The Ozaki scheme as a variable-precision method
- Future work

Conclusion

Everything else

- NVIDIA isn’t the only GPU vendor: AMD’s HPC parts keep strong FP64 throughput
- Is INT8 support here to stay?
- Ozaki-II

References

Bailey, David H. 1995. “A Fortran-90 Based Multiprecision System.” *ACM Transactions on Mathematical Software* 21 (4): 379–87.

Brower, Cole, Samuel Rodriguez Bernabeu, Jeff Hammond, et al. 2026. “Mixed-Precision Ab Initio Tensor Network State Methods Adapted for NVIDIA Blackwell Technology via Emulated FP64 Arithmetic.” *Journal of Chemical Theory and Computation* 22 (13): 6275–82. https://doi.org/10.1021/acs.jctc.6c00203.

Demmel, James et al. 2026. *How to Grade the Accuracy of the BLAS*. arXiv preprint arXiv:2609.12307.

Demmel, James, and Yozo Hida. 2003. “Accurate and Efficient Floating Point Summation.” *SIAM Journal on Scientific Computing* 25 (4): 1214–48.

Fujiwara, Hiroshi. 2010. *Exflib: Extend Precision Floating-Point Arithmetic Library*. Http://www-an.acs.i.kyoto-u.ac.jp/~fujiwara/exflib/exflib-index.html.

Li, Xiaoye S., James W. Demmel, David H. Bailey, et al. 2002. “Design, Implementation and Testing of Extended and Mixed Precision BLAS.” *ACM Transactions on Mathematical Software* 28 (2): 152–205.

Mukunoki, Daichi, Katsuhisa Ozaki, Takeshi Ogita, and Toshiyuki Imamura. 2020. “DGEMM Using Tensor Cores, and Its Accurate and Reproducible Versions.” *High Performance Computing*, 230–48.

NVIDIA. 2025. *Unlocking Tensor Core Performance with Floating Point Emulation in cuBLAS*. Https://developer.nvidia.com/blog/unlocking-tensor-core-performance-with-floating-point-emulation-in-cublas/.

Ogita, Takeshi, Siegfried M. Rump, and Shin'ichi Oishi. 2005. "Accurate Sum and Dot Product." *SIAM Journal on Scientific Computing* 26 (6): 1955–88.

Ootomo, Hiroyuki, Katsuhisa Ozaki, and Rio Yokota. 2024. "DGEMM on Integer Matrix Multiplication Unit." *The International Journal of High Performance Computing Applications* 38 (4): 297–313. https://doi.org/10.1177/10943420241239588.

Ozaki, Katsuhisa, Takeshi Ogita, Shin'ichi Oishi, and Siegfried M. Rump. 2012. "Error-Free Transformations of Matrix Multiplication by Using Fast Routines of Matrix Multiplication and Its Applications." *Numerical Algorithms* 59 (1): 95–118.

Rump, Siegfried M., Takeshi Ogita, and Shin'ichi Oishi. 2008a. "Accurate Floating-Point Summation Part I: Faithful Rounding." *SIAM Journal on Scientific Computing* 31 (1): 189–224.

Rump, Siegfried M., Takeshi Ogita, and Shin'ichi Oishi. 2008b. "Accurate Floating-Point Summation Part II: Sign, K-Fold Faithful and Rounding to Nearest." *SIAM Journal on Scientific Computing* 31 (2): 1269–302.

Schwarz, Angelika, Anton Anders, Cole Brower, et al. 2026. "Guaranteed DGEMM Accuracy While Using Reduced Precision Tensor Cores Through Extensions of the Ozaki Scheme." *Proceedings of the Supercomputing Asia and International Conference on High Performance Computing in Asia Pacific Region (SCA/HPCAsia '26)*, 91–101. https://doi.org/10.1145/3773656.3773670.

The MPFR Library. 2010. Http://www.mpfr.org/.